

Summary for Challenges from Agents to AGI: Data

*Nimet Beyza Bozdag (nbozdag2), Hyeonjeong Ha (hh38),
Shuhaib Mehri (mehri2)*

Problem and Motivation

Access to high-quality data is essential for advancing AI models, but acquiring such data is often costly or impractical. While large language models have shown impressive instruction-following capabilities, they still struggle in unfamiliar environments with ambiguous dynamics. For example, a web-based agent may encounter a previously unseen interface and fail to understand how to interact with it. In such cases, expert-annotated demonstrations would typically be required to guide the agent, but collecting this data for every new environment is both expensive and unsustainable. As a result, without prior experience or annotated data, these agents often generalize poorly and perform unreliably in novel settings. In robotics, the challenge is even greater. Physical agents must interact with the real world, where data collection is slow, error-prone, and resource-intensive. Unlike text or images that can be mined from the web, meaningful robot interactions require trial and error, which is hard to scale without human supervision.

To address these challenges, recent methods like BAGEL and SOAR explore how agents can improve autonomously through synthetic data generation. BAGEL bootstraps digital agents via iterative environment exploration and instruction relabeling using language models, progressively refining behavior without any human supervision. SOAR adopts a similar strategy for robotics, where data collection involves a loop of task proposals and subgoal image generation to form complete trajectories. By using vision-language models to both propose tasks and evaluate success, SOAR enables training goal-conditioned policies directly on self-collected data. Together, these approaches highlight how foundation models and structured self-supervision can drive scalable instruction-following without human-in-the-loop annotations.

Related Works

Instruction-Following Agents

Building agents that can navigate both digital and physical environments through natural language instructions has been a long-standing goal in AI and language understanding [1,2]. Prior work largely relies on expert demonstrations [3, 4, 5] and carefully shaped reward functions [2,3]. In contrast, recent efforts aim to eliminate dependence on demonstrations or explicit rewards by leveraging pretrained models to bootstrap synthetic demonstrations. In the robotics domain, language offers an intuitive interface for instructing robots, with extensive research on language-conditioned policies [6,7], transferring language understanding from pretrained LLMs and VLMs [8,9], and decomposing long-horizon tasks into manageable subgoals [10,11]. Rather than directly conditioning actions on language inputs, recent methods decouple language from motion skills by combining a

language-conditioned subgoal image generator with a goal-conditioned policy. This approach enables self-supervised improvement of motion skills through goal-conditioning, where policies learn from suboptimal trajectories by training to reach hindsight goals [12,13]. Techniques such as image-editing diffusion models (e.g., SuSIE [14]) and goal-conditioned behavioral cloning (GCBC) enable the autonomous improvement of instruction-following capabilities, advancing the development of versatile agents across both digital and physical settings.

LM for Decision Making

Pre-trained language models (LMs) have been increasingly applied to sequential decision-making tasks such as robotic manipulation [7,15], instruction following [16], and tool use [17]. While many approaches rely on human demonstrations [18], either for fine-tuning or as in-context examples for adaptation [16], others integrate LM priors into reinforcement learning frameworks to enhance exploration [19], manage large action spaces [20], or serve as proxy rewards [21]. In contrast, the BAGEL approach eliminates the need for human supervision or reinforcement learning by leveraging purely synthetic demonstrations, combining multiple LM components to bootstrap learning signals autonomously.

Autonomous Improvement of Robot Policies

Recent advances in robot policies have primarily relied on static, high-quality demonstration datasets via imitation learning or offline RL. While some prior work explores fully autonomous policy learning through self-supervised learning or online RL, and others attempt to improve pre-trained policies with autonomous data, these methods often lack improvements in language-conditioned skills and still depend on human demonstrations for bootstrapping. For example, approaches [22] and [23] require hundreds of human demonstrations and task-specific rewards or goal images, while others focus on task-indexed rather than language-conditioned skills. In contrast, our approach autonomously improves language-conditioned skills in a fully self-supervised manner by leveraging a frozen VLM to both select skills for improvement and filter self-collected data for successful outcomes.

Scalable Data and Platforms for Instruction-Following Web Agents

Recent works have advanced scalable data generation and platforms for training web agents beyond BAGEL. OS-Genesis [24] proposes an interaction-driven approach, autonomously exploring GUI environments to synthesize tasks and trajectories, with a reward model ensuring data quality. Synatra [25] transforms indirect knowledge, like tutorials, into structured demonstrations at scale, enabling cost-efficient finetuning of web agents that outperform larger models on web benchmarks. WebOlympus [26] complements these efforts by providing an open, user-friendly platform for running, evaluating, and collecting data from web agents on live websites, featuring safety monitoring and flexible agent integration. Together, these works enable low-cost, scalable training and deployment of instruction-following web agents.

Solution Overview

BAGEL: Bootstrapping Agents by Guiding Exploration with Language.

To enable web agents to generalize to new environments using in-context demonstrations without requiring human supervision, this paper proposes BAGEL: a framework for generating synthetic demonstrations through iterative exploration. These synthetic instruction-trajectory pairs are retrieved at test time based on task similarity and provided to the language model as in-context exemplars.

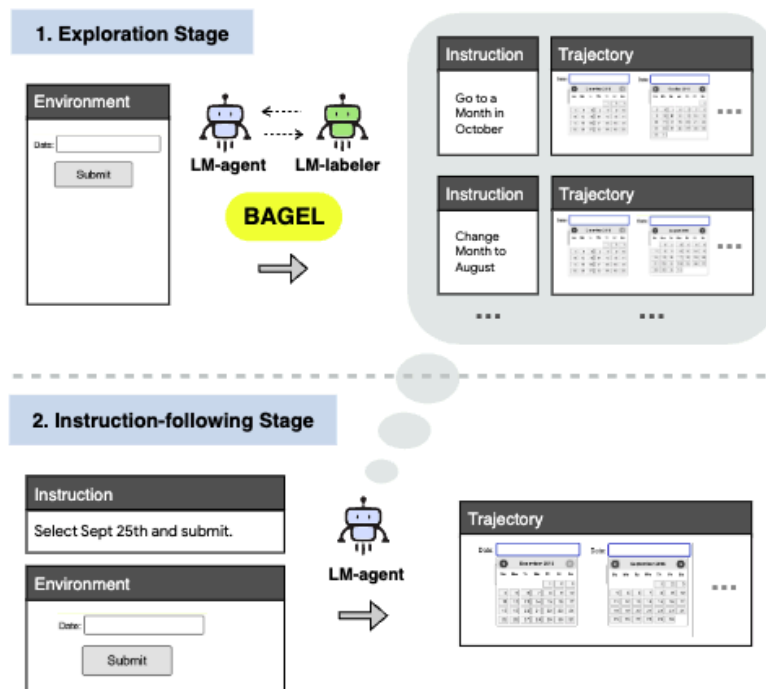


Figure 1. (Top) The exploration stage in BAGEL, which generates synthetic demonstrations through iterative refinement. (Bottom) The LM-Agent is given in-context exemplars from the generated demonstrations.

The core idea behind BAGEL is to bootstrap learning through a cycle of trajectory generation and instruction relabeling, guided by a demonstration filter. In the trajectory-first variant, the agent begins by exploring the environment and generating a trajectory of actions without conditioning on any instruction. A trajectory labeler then produces a corresponding instruction that describes the behavior. The resulting instruction-trajectory pair is evaluated by a demonstration filter, which makes a binary judgment on whether the trajectory successfully executes the given instruction.

Alternatively, in the instruction-first variant, the process begins by sampling a plausible instruction. The agent then attempts to generate a trajectory that completes the task. In both approaches, BAGEL performs iterative refinement, where instructions and trajectories are

repeatedly updated based on each other, guided by the demonstration filter, to improve alignment and quality.

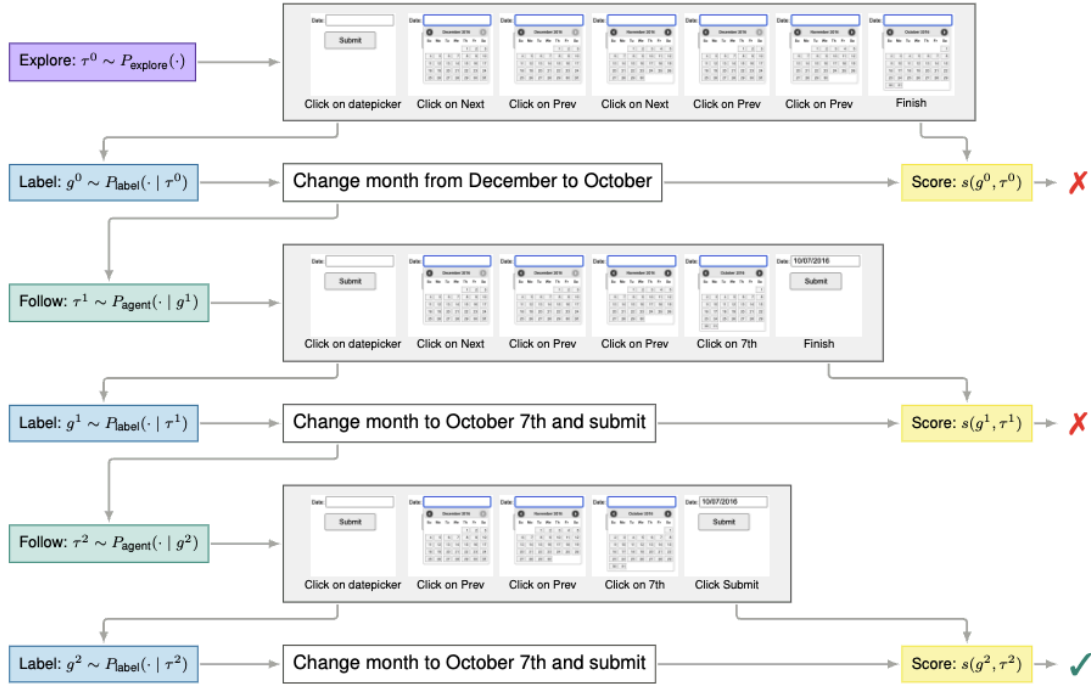


Figure 2. The iterative refinement process of BAGEL. First, a trajectory is sampled from the exploration policy, then an instruction is generated with the trajectory labeler. The pair is then scored by the filter. Both the trajectory and instruction are refined until they pass through the filter.

Over time, BAGEL collects a diverse set of high-quality synthetic demonstrations that align with natural instructions and reflect valid environment dynamics. At test time, relevant demonstrations are retrieved using embedding-based similarity and included in the prompt for in-context learning. This method allows agents to adapt to novel tasks without requiring expert-annotated data, leveraging language model capabilities and environment feedback for self-supervised improvement.

Results: The authors evaluate BAGEL on two benchmarks: MiniWoB++ and ToolQA. On the 10 most challenging tasks in MiniWoB++, BAGEL achieves a 13% improvement over the zero-shot baseline using the trajectory-first variant. In contrast, results on ToolQA show a less impressive 2% improvement with the instruction-first variant. Ablation studies highlight the benefits of BAGEL's iterative refinement process, demonstrating that repeated rounds of instruction and trajectory relabeling lead to higher-quality demonstrations and improved agent performance. Additionally, they show that BAGEL reduces execution failures. In MiniWoB++, the number of execution errors is reduced by up to 13 times compared to the zero-shot baseline, indicating better alignment with environment dynamics and more reliable action sequences.

Autonomous Improvement of Instruction Following Skills via Foundation Models

This work proposes SOAR, an approach that allows autonomous data collection to improve robot capabilities. It addresses two problems (1) automating a scalable data collection procedure that can collect diverse and meaningful data) and (2) learning from non-optimal, autonomous data that doesn't use human annotators. The framework uses environment feedback and vision-language models to generate training data, and enables autonomous data collection without the need for human annotators. In this work, they focus on improving instruction-following abilities.

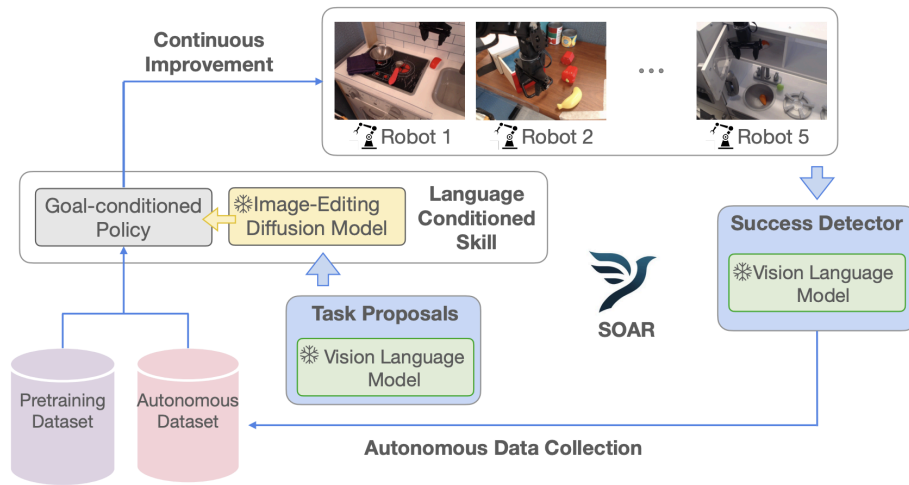


Figure 2: Overview of SOAR autonomous improvement pipeline: First, we equip the robot with a set of basic skill by pre-training. Then, we deploy the pre-trained policy on a fleet of five robots to autonomously collect data, with a VLM proposing viable language tasks to practice. Specifically, the language task is turned into a subgoal image via an pre-trained image-editing diffusion model, and the robot executes a goal-conditioned policy. Finally, we use a VLM to label success information of the collected trajectories, and train the policy using this data, resulting in improvement.

SOAR is built of four key components:

1. **VLM Task Proposals:** A strong vision language model (VLM) observes the environment and proposes a meaningful, candidate task that the robot can physically complete in its environment.
2. **Language-Conditioned Control as Goal-Conditioned Control:** Uses a language-conditioned diffusion model (SuSIE) to transform language instructions into visual subgoal representations. They also use goal-conditioned policy (GCBC) that moves the current state to the generate subgoal image.
3. **VLM Success Detection:** After the task is executed, the VLM inspects the final state and determines whether or not the robot was successful. Successful trajectories are more likely to contain meaningful interaction data, so they use this data to train on.
4. **Policy improvement:** In this stage, we will use the autonomous data to self-improve over the original policy that is used for data collection. We focus on the low-level

goal-condition policy, since that is decoupled from language understanding and is a more accurate representation of policy improvements. Through hindsight relabeling, parts of each trajectory can be converted into goal-reaching examples, even if the final outcome deviated from the initial goal. This leverages unsupervised data collection to continuously improve the robots abilities.

Results: To evaluate there method, the authors ran experiments that evaluate the end-to-end improvement, and compared it against alternative methods. They use five WidowX 250 6-DoF robot arms in their experiments and gather around 120 hours of autonomous data using SOAR. They train on the autonomous data collected (416K transitions) as well as the pre-training dataset (1.87M transitions). The results demonstrate that SOAR significantly increases the average success rate. First, they train on autonomous data from each scene and the success rates improves from 28% to 57%. Then, the train on data from all ten scenes combined, and this reaches 65% average success. They also show that their method outperforms other methods, such as direct language-conditioned behavior cloning, and helps get higher quality data and more effective learning.

Limitations

BAGEL

1. BAGEL relies on imperfect language models, which can produce suboptimal instructions or trajectories. The filtering process may also fail, letting low-quality demonstrations through.
2. Generated demonstrations often lack diversity, which limits the usefulness of retrieved examples at test time and hurts generalization.
3. BAGEL struggles with generating demonstrations for complex or long-horizon tasks, as LMs often fail to produce successful trajectories for them.
4. The limited improvement on ToolQA suggests challenges in scaling BAGEL to more complex, real-world environments.

SORA

1. It only incorporates successful trajectories in it's training, but unsuccessful trajectories also hold valuable training signals that can be leveraged.
2. Their method relies on powerful foundation models (i.e. VLMs), so those will serve as a bottleneck to how effective it can be.
3. They explore a limited set of tasks in this work, it would be interesting to see how effective this framework ins in other tasks (dynamic or dextrous).

Future Research Directions

BAGEL

1. Fine-tuning on Synthetic Demonstrations
 - a. BAGEL currently uses synthetic demonstrations only as in-context

examples. Future work could explore fine-tuning language models on these demonstrations to develop a more internalized and robust understanding of environment dynamics.

2. Conditioned Trajectory Generation
 - a. To improve diversity and coverage, future iterations of BAGEL could condition trajectory generation on specific environmental elements, such as particular HTML tags or UI components. This would encourage exploration of underused areas of the interface and result in a wider variety of instruction-trajectory pairs.

SORA

1. Lifelong Learning and Continual Policy Refinement
 - a. While SOAR collects autonomous data in batches for policy updates, we can investigate continual, online learning frameworks where policies are incrementally updated in real-time during deployment, enabling lifelong skill acquisition and adaptation.
2. Policy Robustness to Noisy Labels and Failures
 - a. While SOAR uses only successful trajectories, it doesn't handle noisy labels robustly. We can explore robust self-supervised learning techniques that can explicitly handle noisy or unsuccessful trajectories, perhaps leveraging self-refinement loops.

Summary of Class Discussion

BAGEL

Q1: Is the 2% improvement over the base LM policy on ToolQA significant enough? Why do you think there is a more significant improvement on MiniWoB++?

A1: 2% is not enough. Web-based task in MiniWoB++ can be easily handled by trajectories generated in BAGEL.

Q2: Why do you think instruction-first and trajectory-first methods yield different results and why is one not better than the other in both cases? For example instruction-first performs 5% worse than trajectory-first for MiniWoB++, but about 1% better in ToolQA.

A2: MiniWoB++ requires longer trajectories while ToolQA might not include trajectory-based tasks.

Q3: How do the authors ensure that the demonstration filtering process correctly filters suboptimal demonstrations?

A3: They don't do filtering process

Q4: Do the synthetic demonstrations contain complex examples, or do the LMs opt for easier demonstrations? If so, how can BAGEL expand to more complex/real-life scenarios?

A4: LMs tend to generate easy trajectories, however, this will give LMs to understand the easy basic blocks on how to interact with this given environment.

SORA

Q1: The paper says they only train on successful trajectories, do you think there would be any benefit in training on both successful and unsuccessful trajectories? What type of training method would you use? How would you change the data generation pipeline?

A1: Incorporating unsuccessful trajectories could boost training.

Q2: How can we go beyond successful/unsuccessful trajectories? (More finegrained evaluations)

A2: For evaluation perspective, # of steps to complete the tasks could be incorporated into the rewards, and evaluate the reward during the inference time.

Reference

1. Allen, James, et al. "Plow: A collaborative task learning agent." *AAAI*. Vol. 7. 2007.
2. Branavan, Satchuthananthavale RK, et al. "Reinforcement learning for mapping instructions to actions." *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP*. 2009.
3. Liu, Evan Zheran, et al. "Reinforcement learning on web interfaces using workflow-guided exploration." *arXiv preprint arXiv:1802.08802* (2018).
4. Humphreys, Peter C., et al. "A data-driven approach for learning to control computers." *International Conference on Machine Learning*. PMLR, 2022.
5. Furuta, Hiroki, et al. "Multimodal web navigation with instruction-finetuned foundation models." *arXiv preprint arXiv:2305.11854* (2023).
6. Stepputtis, Simon, et al. "Language-conditioned imitation learning for robot manipulation tasks." *Advances in Neural Information Processing Systems* 33 (2020): 13139-13150.
7. Ahn, Michael, et al. "Do as i can, not as i say: Grounding language in robotic affordances." *arXiv preprint arXiv:2204.01691* (2022).
8. Achiam, Josh, et al. "Gpt-4 technical report." *arXiv preprint arXiv:2303.08774* (2023).
9. Devlin, Jacob, et al. "Bert: Pre-training of deep bidirectional transformers for language understanding." *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019.
10. Akiyama, Shogo, et al. "Open-loop VLM Robot Planning: An Investigation of Fine-tuning and Prompt Engineering Strategies." *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*. 2024.
11. Kannan, Shyam Sundar, Vishnunandan LN Venkatesh, and Byung-Cheol Min. "Smart-Illm: Smart multi-agent robot task planning using large language models." *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024.
12. Chebotar, Yevgen, et al. "Actionable models: Unsupervised offline reinforcement learning of robotic skills." *arXiv preprint arXiv:2104.07749* (2021).
13. Andrychowicz, Marcin, et al. "Hindsight experience replay." *Advances in neural information processing systems* 30 (2017).
14. Black, Kevin, et al. "Zero-shot robotic manipulation with pretrained image-editing diffusion models." *arXiv preprint arXiv:2310.10639* (2023).
15. Liang, Jacky, et al. "Code as policies: Language model programs for embodied control."

- 2023 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2023.
16. Yao, Shunyu, et al. "React: Synergizing reasoning and acting in language models, 2023." URL <https://arxiv.org/abs/2210.03629> (2023).
 17. Parisi, Aaron, Yao Zhao, and Noah Fiedel. "Talm: Tool augmented language models." arXiv preprint arXiv:2205.12255 (2022).
 18. Nakano, Reiichiro, et al. "Webgpt: Browser-assisted question-answering with human feedback." arXiv preprint arXiv:2112.09332 (2021).
 19. Mu, Jesse, et al. "Improving intrinsic exploration with language abstractions." Advances in Neural Information Processing Systems 35 (2022): 33947-33960.
 20. Yao, Shunyu, et al. "Keep calm and explore: Language models for action generation in text-based games." arXiv preprint arXiv:2010.02903 (2020).
 21. Kwon, Minae, et al. "Reward design with language models." arXiv preprint arXiv:2303.00001 (2023).
 22. Bousmalis, Konstantinos, et al. "Robocat: A self-improving foundation agent for robotic manipulation." arXiv preprint arXiv:2306.11706 1.8 (2023).
 23. Yang, Jingyun, et al. "Robot fine-tuning made easy: Pre-training rewards and policies for autonomous real-world reinforcement learning." 2024 IEEE International Conference on Robotics and Automation (ICRA). IEEE, 2024.
 24. Sun, Qiushi, et al. "OS-Genesis: Automating GUI Agent Trajectory Construction via Reverse Task Synthesis." arXiv preprint arXiv:2412.19723 (2024).
 25. Ou, Tianyue, et al. "Synatra: Turning indirect knowledge into direct demonstrations for digital agents at scale." arXiv preprint arXiv:2409.15637 (2024).
 26. Zheng, Boyuan, et al. "Webolympus: An open platform for web agents on live websites." Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations. 2024.